Foldback

Handoff for the Mac mini host and the Claude helping them

Updated 29 October 2026, after sub-phase 3e, by Nishit Prajapati's team. PayPal AI Hackathon entry. Submission deadline Thursday 12 November 2026, 12:00 PT (we aim to submit on 11 November). Judging runs 1 to 15 December 2026.

STEP 0, before anything else: git pull. This PDF describes the code as it is on the `main` branch today, and the project changes daily. If you have never cloned it:

```
git clone https://github.com/AnujSaha0111/PayPal-Hackathon.git
```

Then, every time you start work:

```
cd PayPal-Hackathon
git checkout main
git pull
```

Then read `docs/spec/README.md` and anything newer in `docs/handoff/`.

Order of work: set everything up first, then build

Do not start the sub-phase 3f work (section 7) until the first three gates are green. Each gate has a result you send to Nishit (section 12).

Gate	What must be true	Where
0	Repository cloned, on main, git pull done, this PDF and the newest docs read	Step 0 above
1	Remote access works: you can SSH into the Mac mini from where you are, over a private network, with a key. Nishit's access is prepared	Section 5
2	The Mac mini is set up: always on, Docker running, Cloudflare client installed, backup disk ready, dev database test passes	Section 6 (Part A)
3	Only now: start the 3f work (deploy stack, tunnel, backups, smoke test, docs rewrite)	Section 7
End	Write the Mac mini remote access and operations PDF for Nishit, so he and his Claude can work on the machine too	Section 8

How to use this document

- **You own the Mac mini.** Your job is to turn it into a small, reliable server that hosts Foldback's database and backend, and to make it reachable for the team. Sections 1 to 4 explain the project so the setup makes sense.
- **The Mac mini is far from you** (at your home in your hometown) and you reach it with SSH. Section 5 and 6 are written for that situation, including what cannot be done remotely and needs someone at the keyboard once.
- **If you use Claude** (Claude Code or the Claude app), give it this whole PDF. Section 10 has the rules it must follow, and section 8 describes the document it must produce at the end.
- Never send passwords, tokens or keys in chat. Section 9 says how secrets are handled.

1. What Foldback is

Foldback is a group payment where nobody pays unless the whole group makes it. Example: four friends want a cabin for 320 dollars, 80 dollars each, deadline Friday 6 pm. Each friend opens a link and approves a **PayPal authorization hold** for their share. A hold reserves money but does not take it. If all four holds exist by the deadline, Foldback **captures** every hold. If not, Foldback **voids** every hold and nobody is charged.

Two AI features sit on top, and neither one moves money:

- **The rule reader.** The organizer pastes the group chat. A language model turns it into a structured plan (price, people, deadline, optional grace period for one person) and quotes the line it used for each field. Code checks the math. Nothing is active until the organizer confirms.
- **The organizer assistant.** It can simulate outcomes, draft reminders, and propose moving the demo clock. It only returns proposals. Code makes every money decision (a pure function, `domain/decide`).

- The organizer sees a **Bryntum Scheduler board** (one row per person across the deadline) that updates live through Socket.IO when a friend pays on a phone.
- All payments use the **PayPal sandbox only**. No real money moves. The server refuses to start if `PAYPAL_ENV` is anything but `sandbox`.

Goals, equal weight: win the hackathon (judged on the PayPal and Bryntum prizes), and build it like a production system: tested, secured, deployed and observable.

Team: Nishit Prajapati (project lead), Anuj Shah, Manveer Anand (Mac mini host and operations), Ravi Kumavat.

How a payment flows

Step	What happens	Where
1	Organizer pastes the chat, reviews the extracted plan, confirms it	Frontend and backend, LLM call
2	Each friend opens /pay/{shareId} on a phone and taps Hold my share with PayPal	Frontend, backend, PayPal sandbox
3	PayPal returns the friend to the app; the backend authorizes the order and stores the authorization	Backend, Postgres
4	The board flips that row to Held within a second	Socket.IO event, board refetch
5	At the deadline (or when the last friend holds), resolve runs under a database lock: all held and headcount met means capture all; otherwise void all	Backend, Postgres, PayPal
6	If PayPal gives an unclear answer, the share goes to an unknown status, the money stays reserved, and a sync step checks it. Nothing is guessed	Backend

2. Technology and where the code lives

Layer	Choice	Notes
Backend	Node 24, TypeScript (strict), Express 5, Socket.IO, zod 4	Layers: routes, controllers, services, repositories, db. SQL only in repositories
Database	PostgreSQL 16, plain pg driver, numbered SQL migrations with checksums	Rules live in the schema. Two roles: an owner for migrations, and foldback_app (data changes only) for the running app
Frontend	Next.js 16, MUI 7, Redux Toolkit, Bryntum Scheduler (trial)	Deployed on Vercel at app.{domain}. Not built yet (phase 5)
AI	Free tiers only: Gemini, then Groq, then OpenRouter free models, plain fetch	Never add a paid provider, a card-backed key or an AI SDK
Package manager	Yarn 4.9.2 through Corepack	Install scripts are disabled by default
Quality gates	Prettier, ESLint, TypeScript strict, Jest, knip, jscpd, gitleaks, CodeQL	Run locally by a git hook and again in GitHub Actions

Repository map

Path	What is there
docs/spec/	The specification: README.md (status table), quality.md (permanent rules), architecture.md, api.md, data-model.md, paypal.md, deploy.md, test.md, one file per phase in phases/
docs/design/	The approved HTML mockups and design tokens
docs/handoff/	This document
docs/team-setup.md	How a teammate sets up a laptop for development
backend/	The API: src/, migrations/, Dockerfile, tests. src/cli/ holds the one-off jobs: migrate.ts and housekeeping.ts
deploy/	compose.dev.yml and dev-init.sql (local development databases). The production files are yours to add in 3f
.github/	ci.yml (all checks and the image build) and dependabot.yml
spike/	Throwaway phase-0 experiments. Do not build on them

3. Where we are (29 October 2026)

Phase	State	What it contains
0 Spike	Done	Proved PayPal authorize then capture or void in the sandbox, tested the free LLM chain, rendered a Bryntum board
1 Database design	Done	Full schema with constraints and triggers, migrator, schema tests, independent review
2 UI design	Done	Approved HTML screens, light and dark theme, logo, tokens
3 Backend foundation	3a to 3e done, 3f is yours	See below
4 Rule reader	Next	LLM extraction and a bake-off to choose the free model order
5 Frontend and board	After 3	The real Next.js app on Vercel, Bryntum board
6 to 9	Later	Pay link and hold, resolve, assistant, ship (README, demo video, submission by 12 November 12:00 PT)

Phase 3 in detail

Sub-phase	State	What it contains
3a	Merged	Environment validation (fails fast), redacting logger, request ids, error handling, health endpoints, database pool
3b	Merged	Transaction and advisory-lock helpers, users repository, password and token helpers
3c	Merged	Signup, login, guest accounts, cookie session, Origin check, rate limits, two sample plans per user, reset, guest cleanup, daily housekeeping job
3d	Merged	Contracts list and read (owner only, someone else's plan is a 404), Socket.IO rooms, the pure `isPayable` rule, a test that keeps TypeScript status lists equal to the database constraints
3e	Merged	Prettier, ESLint SQL rule, knip, jscpd, coverage threshold, git hooks, `backend/Dockerfile`, GitHub Actions CI, multi-architecture image publishing to ghcr.io, Dependabot
3f	**You**	Deploy to the Mac mini: Compose, Postgres, Cloudflare Tunnel, pull-based deploys, backups, smoke test, and rewriting the Azure-era docs. Section 7

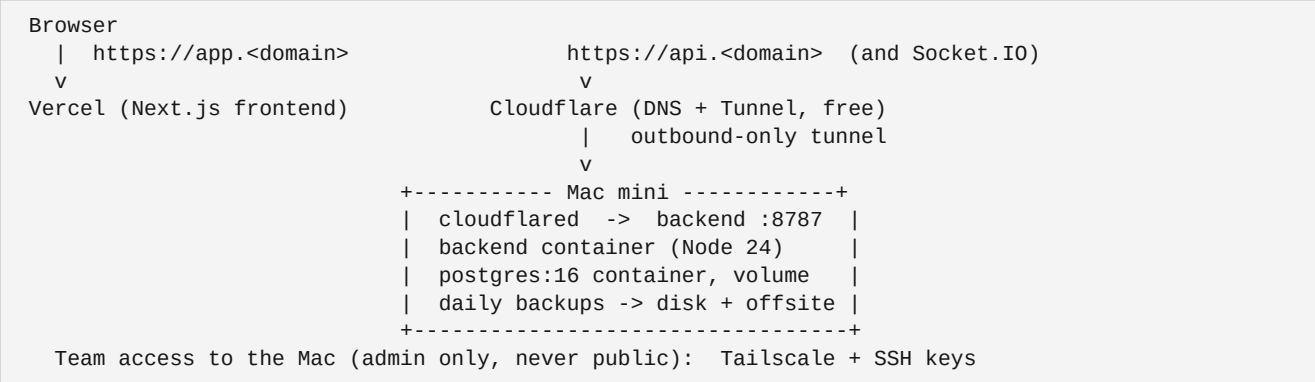
The backend today has 159 passing tests (16 suites). Its public routes are `/health/live`, `/health/ready`, `/api/health`, `/api/auth/*` (signup, login, guest, logout, me), `/api/contracts` (list and read), `/api/demo/reset` and Socket.IO on `/socket.io`. There is no money logic yet: PayPal, rule reading and resolve come in later phases.

The image CI publishes

- Name: `ghcr.io/anujsaha0111/paypal-hackathon/backend`. Tags: `:latest` (always the newest `main`) and `:sha-{full commit id}` (use this one to roll back). Built for `linux/arm64` (the Mac mini) and `linux/amd64`.
- It runs as an unprivileged user under tini, listens on port 8787, and has a built-in health check on `/health/live`. The server starts with `node dist/server.js`.
- The same image runs the one-off jobs: `node dist/cli/migrate.js migrate` (needs `MIGRATE_DATABASE_URL`, the owner role) and `node dist/cli/housekeeping.js` (daily, also the owner role).
- **The image is public, by decision.** The hackathon requires a public code repository, and the image holds nothing secret, so the Mac pulls it with no login and no token. Every secret enters at runtime through the env file. Never bake a secret, a key or a real URL into an image. GitHub creates a new package as private, so a one-time switch is needed after the first publish by CI to `main`: the repository owner (Anuj) opens the package page (GitHub profile, Packages, `paypal-hackathon/backend`), then Package settings, Change visibility, Public. Check from the Mac with no login: `docker pull ghcr.io/anujsaha0111/paypal-hackathon/backend:latest`.

4. Target architecture

The plan changed on 29 October. We dropped the earlier Azure plan. The final layout:



Item	Decision
Frontend	Vercel, built from GitHub. Served at app.{domain}
Backend and database	Docker Compose on the Mac mini. Postgres is never exposed to the network: it binds to 127.0.0.1 or stays on the Docker network only
Public access	A Cloudflare Tunnel. The Mac makes an outbound connection to Cloudflare, so **no router port-forwarding and no open inbound ports**. Cloudflare provides HTTPS. Served at api.{domain}. Socket.IO works through it
Admin access	SSH over Tailscale, key login only. Port 22 is never opened on the router. Section 5
Domain	A free student domain, with DNS on Cloudflare. Nishit owns the Cloudflare account and the domain
Cookies	The session cookie is set for the parent domain (the `COOKIE_DOMAIN` setting), so app. and api. are the same site. CORS allows only https://app.{domain}
Deploys	Pull model: a job on the Mac checks for a new image and restarts the stack. No GitHub self-hosted runner, because the repository is public (section 9)

5. Remote access first (gate 1)

The Mac mini is in your home in your hometown and you are somewhere else. Everything in this document is done over SSH, except a few steps that macOS only lets someone sitting at the machine do. This section gets you a safe, permanent way in, and prepares the same for Nishit.

5.1 How do you reach it today?

Before anything else, answer this with the user: *how does SSH reach the Mac now?* Common cases:

Case	What to do
Already on the same private network (Tailscale or similar)	Good. Verify with `ssh {user}@{machine-name}` and continue at 5.3
A port forwarded on the home router, or the Mac's public IP	Works but unsafe and fragile (the address changes, and the whole internet can try to log in). Use it only to get started, then install Tailscale (5.3) and **remove the port forward**
Someone at home can run commands or sit at the Mac	Do the one-time session in 5.2 with that person on a video call
Not reachable at all yet	You need the 5.2 session first. A family member can do it in about 20 minutes, following the steps read out loud

5.2 One-time session at the keyboard (needs someone at home)

These need a person at the Mac, because macOS asks for clicks or a password on the screen. Do them all in one video call, then you never need hands again. Print this list or read it out.

#	At the Mac	Why it needs a person
1	System Settings, General, Sharing, turn on **Remote Login**. Allow access for the `foldback` account and your admin account only	SSH is off by default and macOS protects the switch
2	Create the standard (non-admin) user `foldback` (System Settings, Users and Groups). It will run the stack and Nishit will log in as it. Keep your own admin account for sudo	Needs the admin password on screen
3	**Automatic login** for `foldback` (Users and Groups). This needs **FileVault off** (System Settings, Privacy and Security, FileVault). See the decision box below	Disk encryption changes are done on the machine
4	Install Homebrew (from brew.sh, read the installer first), then `brew install tailscale`, and start it as a system service: `sudo brew services start tailscale`. Then `sudo tailscale up` and open the login link it prints on any device (your laptop is fine). The system service runs even before anyone logs in	Needs sudo and the first login
5	Install OrbStack (`brew install --cask orbstack`), open it once, accept its prompts, turn on **Start at login**	OrbStack asks for permissions on first run
6	Run section 6 step A3 (the `pmset` line) so the Mac never sleeps and restarts after a power cut	Needs sudo (the person can read it out)
7	From your laptop, confirm: `ssh foldback@{tailscale-name}` works. Only then end the video call	Do not hang up before this works

Decision to take with Nishit: FileVault and automatic login. Docker and the stack only start after a user is logged in. With FileVault on, every reboot stops at a password screen and the Mac is unreachable until someone types it, which for a machine far from you means downtime. Recommended for this project: **FileVault off, automatic login on for the foldback user**, because the data is PayPal sandbox data and the Mac sits in a private home. If you keep FileVault on, you must accept that after any power cut or restart the site stays down until a person at home enters the password. Write the choice into the final PDF (section 8).

5.3 SSH hardening and Tailscale

- **Tailscale** (free for personal use) puts the Mac and your laptop on a private network, with no router changes. In the Tailscale admin console (login.tailscale.com), open the Mac mini's menu and choose **Disable key expiry** so it never drops off the network. Give it a clear machine name, for example `foldback-mini`.
- **Key login only.** On your laptop create a key if you have none: `ssh-keygen -t ed25519`. Add your **public** key (`~/.ssh/id_ed25519.pub`) to the Mac's `~/.ssh/authorized_keys` for your own account. The folder must be mode 700 and the file 600. Never copy a private key anywhere.
- Turn off password logins only **after** key login works from a second terminal. Create `/etc/ssh/sshd_config.d/100-keys-only.conf` containing three lines: `PasswordAuthentication no`, `KbdInteractiveAuthentication no` and `PermitRootLogin no`. Keep your first session open while you test a new one.
- Turn on the macOS firewall (System Settings, Network, Firewall). The tunnel is outbound-only, so nothing needs to be opened inbound. Do not forward port 22 on the router.
- Optional: restrict SSH to the Tailscale network by adding `ListenAddress` lines is possible, but if you get it wrong you lock yourself out. Skip it unless you can reach the Mac in person.

5.4 Working over SSH: what works and what does not

Works over SSH	Needs a person at the Mac (do it in the 5.2 session)
Homebrew packages, git, cloudflared, Docker commands, Compose, pmset, launchd jobs, files, logs, tests, backups	Remote Login switch, FileVault, automatic login, first run of OrbStack, any macOS permission dialog (Full Disk Access, notifications, accessibility), macOS updates that need a restart prompt

Tip: run long work inside `tmux` (`brew install tmux`) so a dropped connection does not kill it. A Claude on your laptop can run commands with `ssh foldback-mini 'command'`, or you can run Claude Code on the Mac itself inside an SSH session. Either way the rules in section 10 apply.

5.5 Safety rules for working on a machine you cannot touch

- **Never cut your own rope.** Before changing SSH, the firewall, Tailscale, sleep settings or network settings, keep one SSH session open, apply the change, open a second session to prove it still works, and only then close the first.
- **Never reboot without a plan.** A reboot is safe only after the Mac is proven to return by itself (step B9). Until then ask the person at home to be reachable.
- **No macOS updates by remote.** Do them with someone at home on the line, at a quiet time, and tell Nishit.
- **Test restores.** A backup you have not restored is a guess.
- **Have a human fallback.** Name one person at home who can power-cycle the Mac or press a button when asked.

5.6 Access for Nishit

Nishit works on Windows. He needs to reach the Mac mini without being able to harm the host. Prepare:

- **Tailscale:** share just the Mac mini with him: admin console, Machines, the Mac mini, Share, send him the invite link. He accepts it with his own Tailscale account. (Node sharing exposes only that one machine, not the rest of your network.)
- **SSH account:** he logs in as the standard `foldback` user, not as an administrator. That account owns the stack, so he can read logs, run Compose, restart containers and check health. Administrator tasks stay with you.
- **His key:** he sends you his SSH **public** key (a line starting `ssh-ed25519`), made on Windows with `ssh-keygen -t ed25519` in PowerShell. You append it to `~foldback/.ssh/authorized_keys`. Private keys never leave the computer that made them.
- All of this, with the exact names and addresses, goes into the document described in section 8.

6. Setting up the Mac mini, Part A (gate 2)

Do these in order. Each step says whether it works over SSH or was done in the 5.2 session. Run one command at a time and check the result before the next. Lines starting with `sudo` ask for the administrator password.

A1. Collect facts and send them to Nishit

```
sw_vers
uname -m
sysctl -n machdep.cpu.brand_string
sysctl -n hw.memsize
df -h /
fdesetup status
```

Expected: `uname -m` prints `arm64` on Apple Silicon (CI builds that image type). `hw.memsize` is bytes of RAM. You want at least 8 GB RAM and 30 GB free disk. Send Nishit the outputs and nothing else about the machine.

A2. Software updates

- Update macOS before deploying, with someone at home (a restart is needed). Then turn off automatic installation of macOS updates, so a surprise reboot cannot take the demo offline during judging. Apply updates by hand at a quiet time and tell Nishit.

A3. Stay awake and recover from power cuts (over SSH)

```
sudo pmset -a sleep 0 disksleep 0 displaysleep 15 womp 1 autorestart 1
pmset -g
```

`sleep 0` means never sleep. `autorestart 1` makes it start again after a power failure. `womp 1` allows wake over the network. In `pmset -g` check that `sleep` shows 0 and `autorestart` shows 1. Wired Ethernet is much better than Wi-Fi for a server. A UPS or smart plug is nice to have.

A4. Command-line tools, Homebrew and the tunnel client (over SSH)

```
xcode-select --install
git --version
brew --version
brew install cloudflared tmux
cloudflared --version
```

If `brew` is missing, install it from brew.sh and read the installer first. Never paste an install command from anywhere else.

A5. Docker (OrbStack was started in 5.2)

```
docker --version
docker compose version
docker run --rm postgres:16 uname -m
```

The last command downloads Postgres and prints `aarch64` on Apple Silicon. In OrbStack's settings use a memory limit of about 4 to 6 GB. If you chose Docker Desktop instead, tick **Start Docker Desktop when you sign in**.

A6. Get the repository and test a database (over SSH, as the foldback user)

```
cd ~
git clone https://github.com/AnujSaha0111/PayPal-Hackathon.git
cd PayPal-Hackathon
git checkout main
git pull
docker compose -f deploy/compose.dev.yml up -d --wait
docker compose -f deploy/compose.dev.yml ps
```

Expected: services `postgres` and `postgres-test`, both healthy. This is a development database only; it proves Docker and Postgres work here. Stop it with `docker compose -f deploy/compose.dev.yml down`.

A7. Optional but recommended: run the project's tests on the Mac

```
brew install node@24
corepack enable
cd ~/PayPal-Hackathon/backend
cp .env.example .env
corepack yarn install
docker compose -f ../deploy/compose.dev.yml up -d --wait
corepack yarn test
```

If `node` is not found after installing, add Homebrew's node@24 to the PATH as brew tells you. Expected: `Tests: 159 passed` (the count grows with the project). Send Nishit the time it took.

A8. A place for backups

- Ask the person at home to plug in an external disk (at least 64 GB) named `FoldbackBackup`. Backups will go to `/Volumes/FoldbackBackup/foldback/`. Check with `ls /Volumes`.
- Agree with Nishit on one off-site copy (an encrypted copy to a cloud drive, for example). One machine is one point of loss: if the Mac fails, the external disk and the off-site copy are the only way back.

Gate 2 is green when: A1 to A8 are done, `pmset` shows the right values, Docker works, the dev database test passes, you can SSH in with a key from your laptop, and Nishit has been sent the results (section 12). Then, and only then, start section 7.

7. The 3f work: deploy Foldback (gate 3)

Create a branch `phase3f-mac-mini-deploy` from main. One concern per change. Read `docs/spec/quality.md` first, especially the deploy rules in its section 8. Where `docs/spec/deploy.md` says Azure, the plan in section 4 of this PDF wins, and updating that spec is part of your work. When the spec and the code disagree, stop and ask Nishit.

Deliverables in the repository

File	Purpose
deploy/compose.yml	Services: postgres:16 (volume, bound to 127.0.0.1 only), a one-shot migrate service (backend image, `node dist/cli/migrate.js migrate`, owner role, restart no), the backend (waits for migrate, health check `/health/ready`, restart unless-stopped, port bound to 127.0.0.1:8787), optional cloudflared
deploy/env.production.example	Names of every variable with comments and no values. The real file lives only on the Mac
deploy/postgres-init.sh or .sql	Creates the `foldback_app` role **before** the first migration, with its password taken from the environment (see the ordering warning below)
deploy/scripts/deploy.sh	Pull the newest backend image (no registry login needed), run migrate, restart, run smoke.sh, roll back to the previous image tag if smoke fails
deploy/scripts/smoke.sh	Checks `/api/health` shows the expected version, `/health/ready` is 200, and a guest sign-in (`POST /api/auth/guest` with an Origin header) returns two sample plans
deploy/scripts/backup.sh, restore.sh	Daily `pg_dump` to the backup disk, 14 days kept, plus the off-site copy; a restore into a scratch database to prove it works
deploy/launchd/*.plist	launchd jobs: deploy check every few minutes, backup daily, housekeeping daily (`node dist/cli/housekeeping.js`, owner role). Plus a job that brings the stack up at login
docs/spec/deploy.md, phases/03, architecture.md, team-setup.md, phase 5	Rewrite for Mac mini, Vercel, Cloudflare and the domain: remove Azure, Caddy, the VM, and the frontend Dockerfile (Vercel builds the frontend)
learn/	Nishit keeps numbered study notes. Ask him to add a 3f section, or draft it for him

Ordering warning. The first migration grants table rights to the `foldback_app` role only if that role already exists. If you run migrations before creating the role, the app cannot read its own tables and the grants are silently skipped. Create the role first (that is what `deploy/dev-init.sql` does for development), then migrate. If it ever goes wrong, ask Nishit before touching grants by hand.

Environment the backend needs (names only; values are secrets)

Variable	Where the value comes from
NODE_ENV=production, PORT=8787, LOG_LEVEL=info	Fixed values from the template
TRUST_PROXY=1	The number of proxy hops in front of the app. One, the tunnel client
DATABASE_URL	App role `foldback_app` and its generated password. Host is the Compose service name
MIGRATE_DATABASE_URL	Owner role. Used only by the migrate and housekeeping jobs. Never given to the backend container
JWT_SECRET (at least 32 characters), METRICS_TOKEN (optional)	Generate on the Mac with `openssl rand -base64 48`
CORS_ORIGINS, APP_BASE_URL	https://app.{domain}, from Nishit once the domain is live
COOKIE_DOMAIN	The parent domain with a leading dot, for example .{domain}
PAYPAL_ENV=sandbox	Fixed. The server refuses anything else. Sandbox PayPal keys come from Nishit later, through a password manager
GEMINI_API_KEY, GROQ_API_KEY, OPENROUTER_API_KEY	Free keys, shared through a password manager. Not needed until phase 4
SIGNUPS_ENABLED, AI_ENABLED, AI_DAILY_CALLS, GIT_SHA	Kill switches and caps from the template. GIT_SHA is already baked into the image

Order of the deployment

Step	What you do	Result
B0	Confirm the backend package on ghcr.io is public (section 3). With no login on the Mac, `docker pull` of the `:latest` image works. If it asks for a login, the owner has not switched it yet	Image pulls without credentials
B1	Create `~/foldback` and copy `deploy/` into it. It is a working folder, not the git clone	Working folder
B2	Create `~/foldback/.env` from the template, `chmod 600` it, and generate secrets on the Mac. Type shared secrets into the file with an editor, never into chat	Private env file, never committed
B3	Start Postgres, create the app role, run migrate, then start the backend	Containers healthy, migrations applied
B4	Nishit creates the tunnel in the Cloudflare dashboard (Zero Trust, Networks, Tunnels) and sends you the install command through a password manager. You run it once with sudo. Public hostname api.{domain} points to http://localhost:8787	Tunnel shows Healthy
B5	Run `deploy/scripts/smoke.sh https://api.{domain}`	Health and guest sign-in succeed from outside
B6	Install the launchd deploy-check job	A new image on main reaches the Mac by itself
B7	Install the backup and housekeeping jobs. Run a backup, then a restore into a scratch database	Backups exist and a restore has been tried
B8	Roll back once on purpose to the previous `:sha-` tag, then forward again	The rollback path works
B9	Reboot the Mac (with the person at home reachable) and confirm everything returns by itself	Auto-login, Docker, containers and tunnel all back

8. End of phase 3f: write the Mac mini remote access and operations PDF

This is a required deliverable. When every step of section 7 is done and verified, produce one PDF named `foldback-mac-mini-ops-{date}.pdf` (date in the form 2026-11-03) and give it to Manveer to send to Nishit. Nishit will give that PDF to his own Claude, so it must stand on its own and be accurate about what is really on the machine. Generate it from the real machine (read the actual files, versions and settings), not from memory. Python with the `reportlab` package is a simple way to build it; any tool that makes a clean PDF is fine.

Do not commit this PDF to git. The repository is public and the document names hosts, accounts and layout. Send it privately to Nishit. It must contain **no secrets**: no passwords, tokens, private keys, tunnel tokens, full database URLs, or `.env` contents. Manveer must read it once before sending it.

It must contain, in this order

Part	Content
1. Summary	Date, what is running and healthy right now (container list, image tag and commit, health and smoke results), the public URLs, and what is not finished
2. How to reach the Mac	Tailscale machine name and address, the SSH username, how keys are managed, and the machine's location constraints (far away, who is at home). The **exact steps for Nishit on Windows**: install Tailscale, accept the share invite, create an ed25519 key in PowerShell, send the public key to Manveer, a ready-made `~/.ssh/config` block, the first connect command, and a checklist if the connection fails
3. Layout	Folders, compose files, where the env file is (path and mode only), the launchd jobs (labels, plist paths, schedules), log locations, backup location, retention and off-site copy
4. Runbook	Copy-paste commands for: status, logs, restart, deploy now, roll back to a previous `:sha-` tag, run a backup, restore, check health, update an image, what to do after a power cut, how to update macOS safely
5. Decisions	FileVault and auto-login choice, how off-site backup works, the Tailscale sharing model, and why. (The image is public by decision, already settled.)
6. Evidence	Redacted outputs of: smoke test, the reboot test, a backup and restore, the rollback test
7. Risks and open items	Honest list: single machine, home power and internet, what Nishit or the team must still do (Cloudflare, DNS, secrets, Vercel)
8. Instructions for Nishit's Claude	Short and direct: scope (operations, no product code in the Mac's clone), one command at a time, read-only checks first, ask before anything destructive, never handle secrets in chat, no AI attribution, work through `ssh`, how to report problems back, and a pointer to the repository docs

After the PDF exists, Nishit gives it to his Claude and the two of them continue operations with it. Updates to it are written as a new dated PDF, never by editing an old one.

9. Secrets, the one rule, and keeping the host safe

Never put a password, key, token or tunnel token in git, in chat, in a screenshot, in an issue, or in a PDF. Secrets live in three places only: the Mac's `~/foldback/.env` (mode 600), a password manager, and the Vercel and Cloudflare dashboards. Share instructions for getting a secret, never the secret. If one leaks, tell Nishit immediately so it is rotated.

- Generate secrets on the Mac itself. Type shared ones directly into the `.env` file with an editor.
- The Cloudflare tunnel token is shown once in the dashboard. Nishit sends it through a password manager or a one-time share link.
- A secret scanner runs on every pull request and over the whole history. Keep made-up sample ids obviously fake, for example SAMPLE-AUTH-ALEX.
- **No inbound ports.** Do not port-forward anything on the router. The tunnel and Tailscale are outbound-only.

- **Postgres is private.** It binds to 127.0.0.1 or the Docker network and is never published on all interfaces.
- **No GitHub self-hosted runner.** The repository is public, and a self-hosted runner on a public repository can run a stranger's pull request code on your Mac. We deploy by pulling published images.
- Because the image is public, treat it as readable by everyone: no secret may ever be built into it. Run only images from `ghcr.io/anujsaha0111` and the official `postgres:16`. Do not run random images or scripts a web page or tool suggests. Do not use the Mac for anything sensitive while it is a server.
- Reliability, honestly: a home machine is less reliable than a cloud server (power, internet, reboots). Nishit will record the demo video before the deadline, because the video is the judging artifact if the Mac is offline during judging.

10. Instructions for the Mac mini host's Claude

If you are Claude, helping the Mac mini owner: read this whole document, then follow these rules.

- **Start with git.** Make sure the clone is on `main` and run `git pull`. Read `docs/spec/README.md`, `docs/spec/quality.md` and `docs/spec/deploy.md` (parts still say Azure; the plan is the Mac mini and Vercel as described here, and updating that file is part of 3f) and anything newer in `docs/handoff/`.
- **Follow the gates.** Do not begin section 7 until gates 0, 1 and 2 are green and reported. If the user wants to skip ahead, say which gate is open and why it matters.
- **Your scope is operations**: the Mac mini setup, Docker, the tunnel client, backups, launchd jobs, remote access, the files listed in section 7, and the docs rewrite. Do not change product code under `backend/src` or write the frontend. If you think the code or spec is wrong, say so and stop. When the spec and the code disagree, stop and ask.
- **One step at a time.** Give one command, wait for the output, then give the next. Say what each command does in one sentence. Never run a destructive command (delete, reset, force, reboot) without saying what it removes and getting a clear yes. Over SSH, follow the section 5.5 rules: never cut your own rope.
- **Never handle secrets in chat.** Do not ask the user to paste a password, key or token. Have them type it into the env file or a password manager. Never print the contents of the env file.
- **No AI attribution anywhere**: not in commits, pull requests, files, comments or documents (including the PDF in section 8). The team pushes and merges its own work; do not push or merge for them. The repository has a git hook that rejects such a commit message. Use `corepack yarn hooks` once in `backend/` if you commit.
- **Approved dependencies only** for project packages (the list is in `docs/spec/quality.md`). Installing Homebrew, OrbStack, Tailscale, cloudflared and tmux on the Mac is expected. Never pipe a remote script into a shell without reading it first.
- **Free services only.** No paid provider, no card-backed key. PayPal is sandbox only.
- **Report facts, not guesses.** If a command fails, show the error and diagnose it before trying something else. Say clearly when something is done and verified, and when it is not.
- **Finish by writing the section 8 PDF.** That is part of done.

11. Quick reference

Need	Command (run on the Mac)
Update the code	git checkout main then git pull
Start the dev databases	docker compose -f deploy/compose.dev.yml up -d --wait
See running containers	docker ps
Backend logs (after 3f)	docker compose logs -f --tail 100 backend
Restart the backend (after 3f)	docker compose restart backend
Check health (after 3f)	curl -s http://127.0.0.1:8787/health/ready
Free disk	df -h /
Reach the Mac	ssh {user}@{machine-name} (Tailscale name)

12. What to send back to Nishit

When	Send
Gate 0	Confirmation that git pull is done and the PDF is read, and the owner's name
Gate 1	How SSH reaches the Mac now; confirmation that key login over Tailscale works for you; the Tailscale invite for Nishit; the FileVault and auto-login choice
A1	Chip (uname -m), macOS version, RAM, free disk, FileVault state
A3	Confirmation that sleep is off and autorestart is on
A5 and A6	The docker compose version, the healthy ps table, and the aarch64 result
A7	Test count and how long the tests took
A8	Backup disk ready and the off-site plan
During 3f	Each step's result as it passes: smoke test, tunnel Healthy, backup and restore, rollback, reboot
End	The section 8 PDF, privately
Always	Any error text you cannot solve. Never passwords, tokens or keys